

vLLM Ascend 推理加速技术原理与落地

王玺源、郑守建



Content 目录

01 vLLM Ascend 项目介绍

02 新特性介绍

03 Decode 显存优化技术

04 展望



Part 01 LLM Ascend 项目介绍



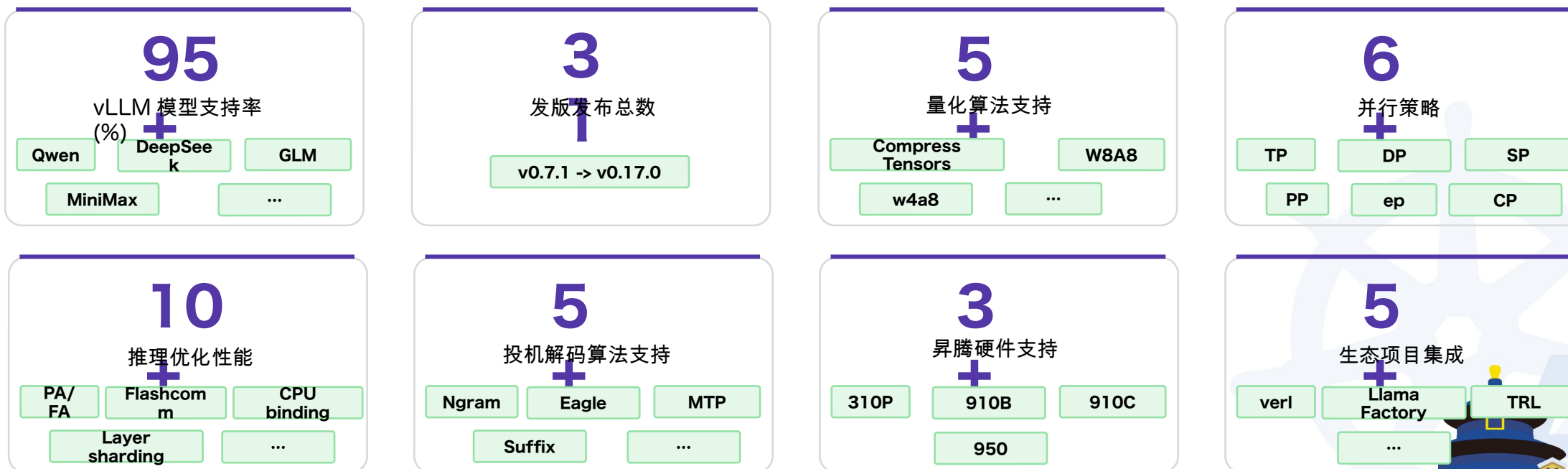
vLLM Ascend: 构建易用、好用的 vLLM 昇腾支持平台



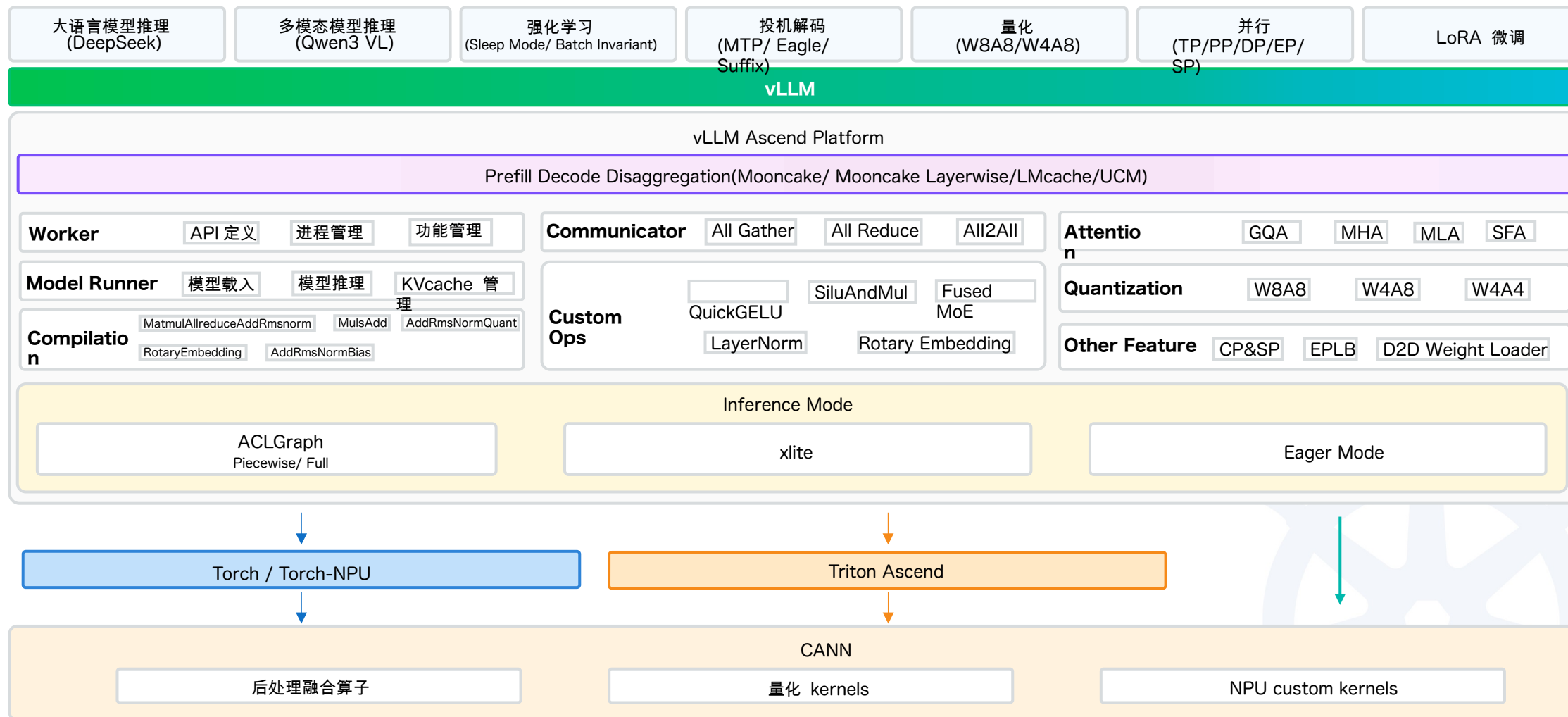
发展历程



项目核心数据



vLLM Ascend: 架构更解耦、泛化能力更强



Part 02

新特性介绍



投机解码整图模式性能进一步提升

vLLM 原生实

仅
PIECEWISE

- 图形模式：V1 仅支持 PIECEWISE 模
- 实现方式：使用 CudagraphDispatcher 进行分片图调度
- 图捕获：即使配置为 FULL 模式，也会降级为 PIECEWISE

vLLM Ascend 实

整图 FULL

- 图形模式：完整支持 CUDAGraphMode. FULL 模式
- 参数更新：_update_full_graph_params() 动态更新图参
- 图捕获：真正的整图模式，无需降级

技术领先性总结

FULL

整图模式支持

合并图

多轮推理合并成一张图

+20%

Decode 性能

vLLM Ascend 核心

优势

✓ 完整的 ACL Graph 支

ACLGraphWrapper 类提供了完整的昇腾 NPU 图捕获和回放能力。支持 FULL 和 PIECEWISE 两种模式，通过 runtime_mode 进行运行时调度。图捕获时使用 Python weakref 弱引用管理图对象，避免循环引用导致的内存泄漏。
对比：vLLM 原生 Eagle 强制降级为 PIECEWISE，无法使用 FULL 模式。

✓ 动态图参数更新

在运行时动态更新注意力参数。支持多步注意力元数据 (multi_steps_attn_metadata) 的批量更新。通过 update_full_graph_params() 函数与后端实现解耦。
对比：vLLM 原生无此机制，PIECEWISE 模式需要多次图调度。

✓ 投机解码深度集成

AscendEagleProposer 类深度集成投机解码与全图模式。支持 Eagle/MTP 等多种投机算法的整图捕获。在图回放时自动处理注意力状态更新，无需额外同步。

对比：vLLM 原生 Eagle 使用 CudagraphDispatcher 分片调度，开销更大。

✓ 性能提升

全图模式减少了 CPU-GPU 同步开销，显著提升吞吐量。单次图捕获覆盖完整前向传播，减少调度开销。针对昇腾 NPU 优化的图执行策略，性能优于原生 PIECEWISE 实现。

对比：vLLM 原生 PIECEWISE 需要多次图切换，性能受限：在典型场景下吞吐量提升约 10%。

NpuGraph-ex 图融合技术，性能提升，泛化性增强

核心定位与优势

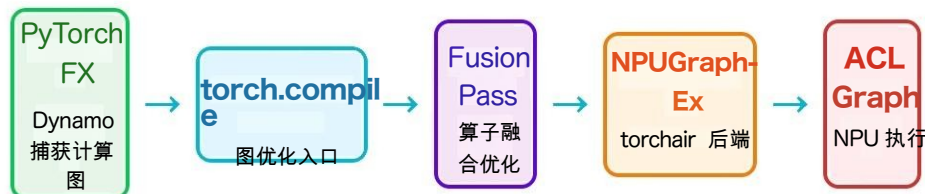
◆ 什么是 NPUGraph-Ex?

- 定义：昇腾 NPU 基于 torch.compile 的整图 ACLGraph 加速方案
- 执行模式：ACLGraph 捕获模式，将模型捕获为单一计算图优化执行
- 核心优势：避免 FX→Ascend IR 多次转换，端到端图优化

◆ 核心功能与进阶特性

- FX** 图优化：减少内存搬运，提升性能
- 内存复用：ACLGraph 间内存复用，降低开销
- 静态 **Kernel**：预编译特定 shape 内核
- 编译缓存：缩短首次推理时延
- 多流表达：多 stream 划分提升并行效率

编译流程



算子融合

Norm + Quant

Add+RMSNorm+Quantization 减少中间内存访问

Matmul+AR+RMS

矩阵乘 + AllReduce + 归一化两阶段通信

QKNorm + RoPE

QK Normalization + Rotary 优化 Attention

Muls + Add

标量乘 + 加法基础融合

编译配置

◆ AscendCompilationConfig 配置选项

enable_npu_graph_ex
启用 NPUGraph-Ex (默认: True)

enable_static_kernel
静态内核编译 (默认: False)

fuse_norm_quant
Norm+Quant 融合 (默认: True)

fuse_qknorm_rope
QKNorm+RoPE 融合 (默认: True)

fuse_allreduce_rms
AR+RMSNorm 融合 (默认: False)

fuse_muls_add
Muls+Add 融合 (默认: True)

静态内核编译模式

- **enable_static_kernel=True** 预编译特定 shape 内核
- 基于 **cudagraph_capture_sizes** 配置
- 避免运行时动态编译开销
- 适用于形状稳定场景

全面适配 Model Runner V2

vLLM Model Runner V2 设计目标

- 完全重构：解决 V1 架构缺陷，代码更清晰
- 异步优先：CUDA/NPU 流执行，CPU 不阻塞 GPU
- 设备原生：Triton 内核处理数据，UVA 零拷贝访问
- 模块化：状态与输入分离，消除并发冲突
- 高效采样：Triton 原生采样器，性能更优

vllm-ascend 适配实现

- NPUModelRunner: 适配昇腾 NPU，重写核心执行流程
- AscendInputBatch: 扩展输入批次，支持 NPU Attention
- AclGraphManager: ACL 图优化，支持全图捕获模式
- AscendBlockTables: 适配 NPU 显存管理
- 投机解码：支持 Eagle/MTP 加速采样

持久化批处理

- 解耦设计：请求状态与输入数据分离
- Gather 操作：GPU 高效收集数据，无需重排
- 永久行分配：请求固定位置，简化调度

异步优先设计

- CUDA/NPU 流：设备自主执行，无 CPU 等待
- 工作队列：CPU 预准备下一步，设备执行当前步
- 重叠执行：最大化设备-CPU 并行

竞态条件消除

- 消除竞争：分离缓冲区
- 实现：CPU 写普通内存，设备读 pinned 内存

分阶段张量写入

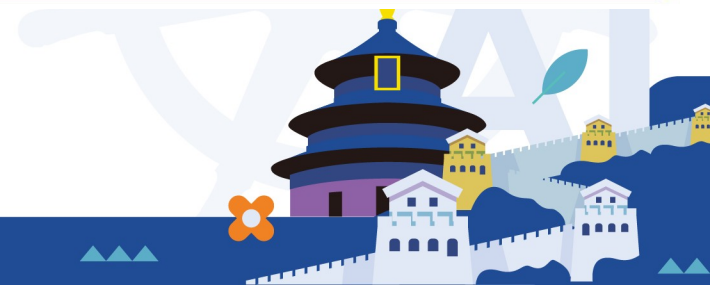
- 场景：块表等大型张量更新
- 步骤：基量在设备→差异在 CPU→打包→单次拷贝
- 优势：减少数据传输，支持灵活更新

设备原生处理

- Triton 内核：GPU/NPU 直接准备输入数据
- UVA：统一虚拟寻址，设备直访 CPU 内存
- 无复制：避免冗余数据传输

Triton 原生采样器

- Gumbel 采样：无需 softmax，内核内随机采样
- Top-K Logprobs：先选 top-k，再计算概率
- 投机解码：索引映射，避免状态膨胀



Part 03

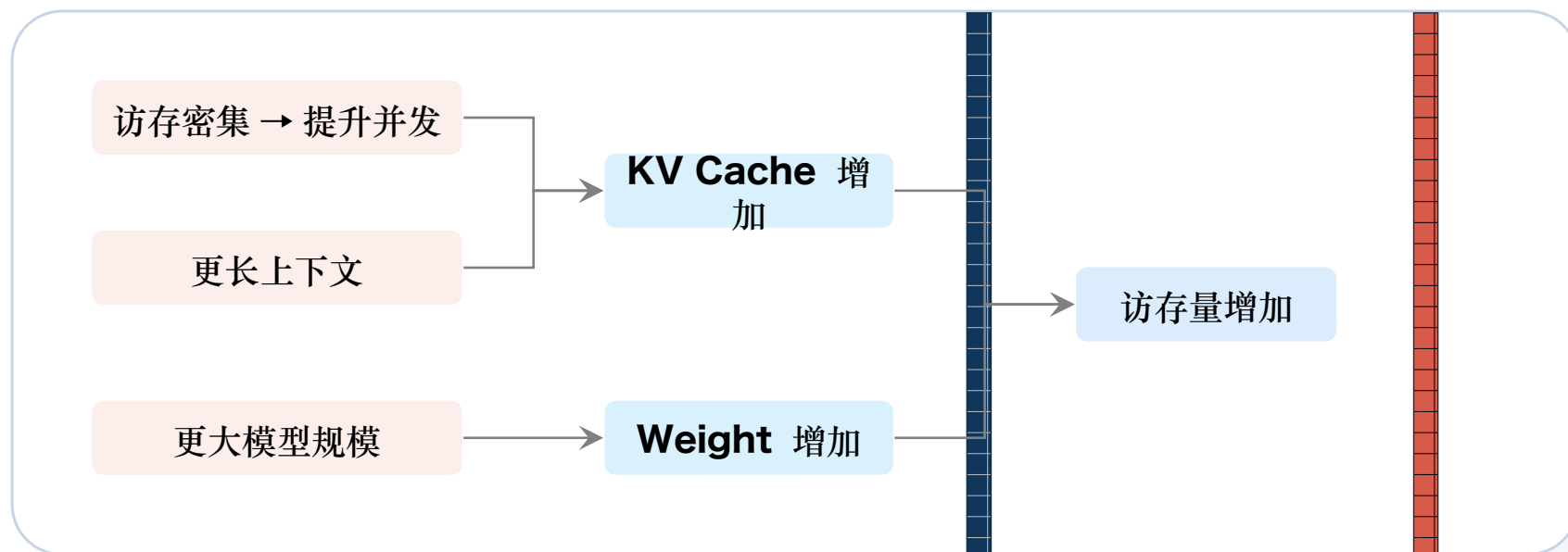
Decode 显存优化技术



Decode 阶段撞上显存墙

问题一：显存容量墙

问题二：访存带宽墙



显存空间 = 权重 + KV Cache 空间 + 激活值 + Buffer

局部异构并行：在显存热点做局部 TP

问题

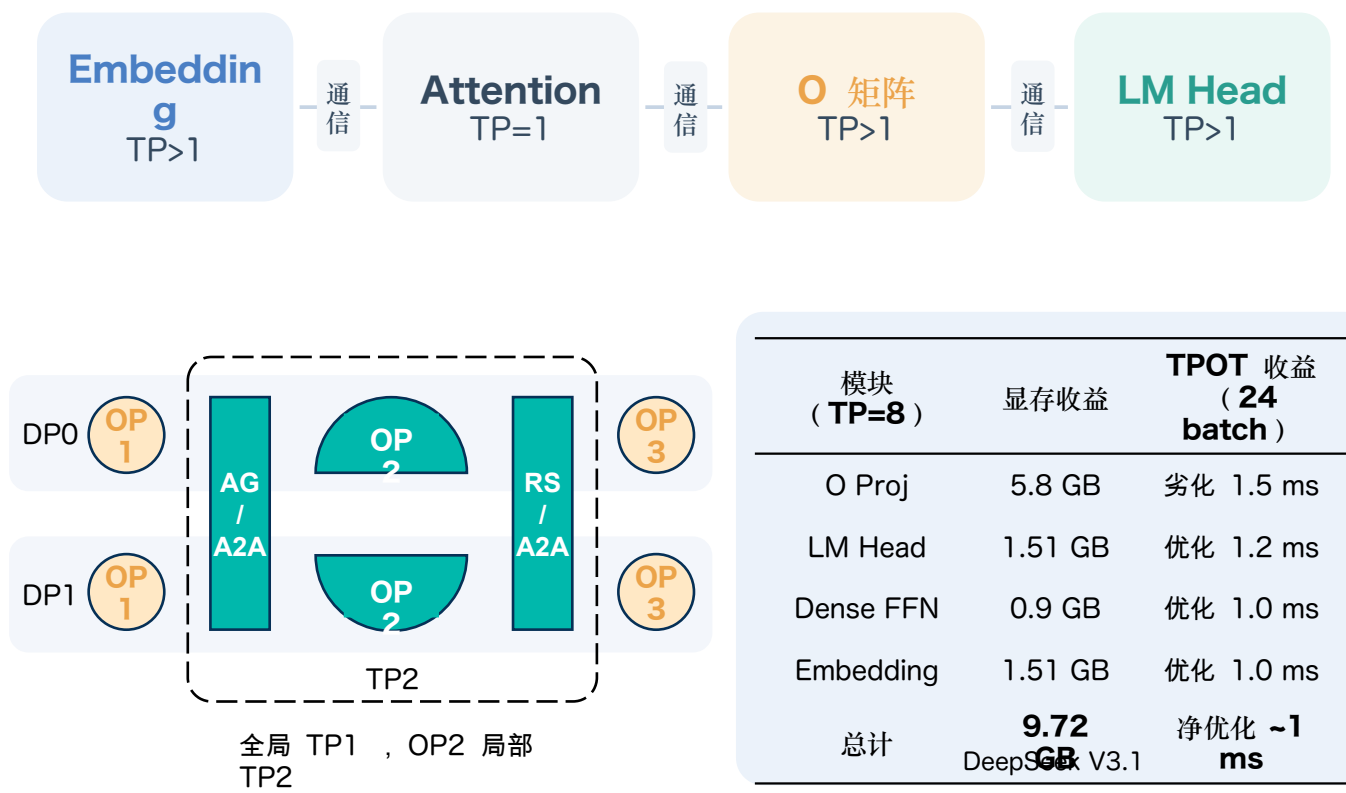
Decode 节点在 TP1 配置下，面临权重显存占用与访存开销双高问题，限制了并发和效率

思路

小通信换大显存，对权重较大的模块进行局部 TP 切分，通过精细的通信来重组激活值

效果

1. 热点模块显存占用降低到 $1/TP$
- 2 时延降低，通信开销小于访存收益



量化：低精权重与 KV

问题

模型规模与序列长度持续增长，HBM容量成为系统瓶颈，影响部署成本与吞吐性能

思路

权重量化与 **KV Cache** 量化降低显存占用与访存量，扩展长上下文能力

效果

1. 相同资源可以支持更大的模型、更高的并发，性能提升 **10%** 以上
2. 数据集分数下降小于 1%

Precision W4A8/W8A8/W4A16/W8A16/...

Granularity per-tensor/per-channel/per-group

Activation Strategy static/dynamic

KV Cache 8-bit GQA/MLA...



DCP：按序列维分散存储 KV Cache

背景

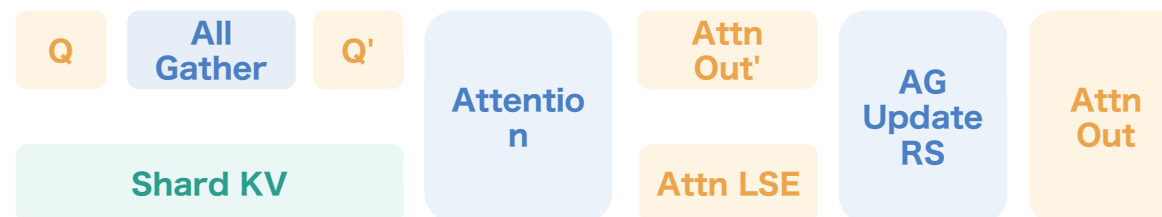
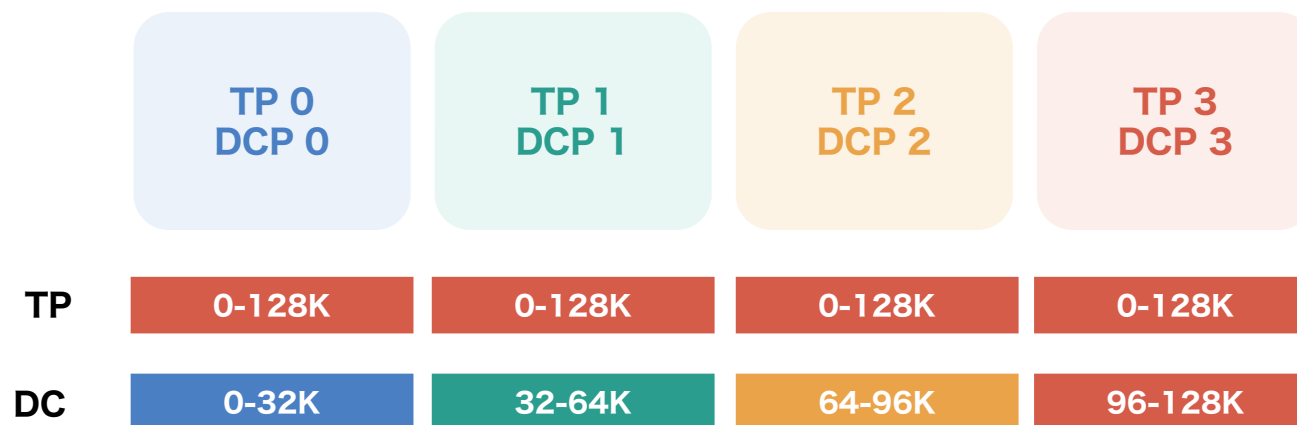
长上下文是大模型发展趋势之一，这也意味着所需的 KV Cache 空间越来越大

思路

1. 增大 TP 降低权重占比
2. DCP 域内按序列维度分散存储 KV Cache

效果

上下文长度可以扩展到 **DCP** 倍以上



Weight Prefetch : 流水重叠掩盖访存延迟

机会

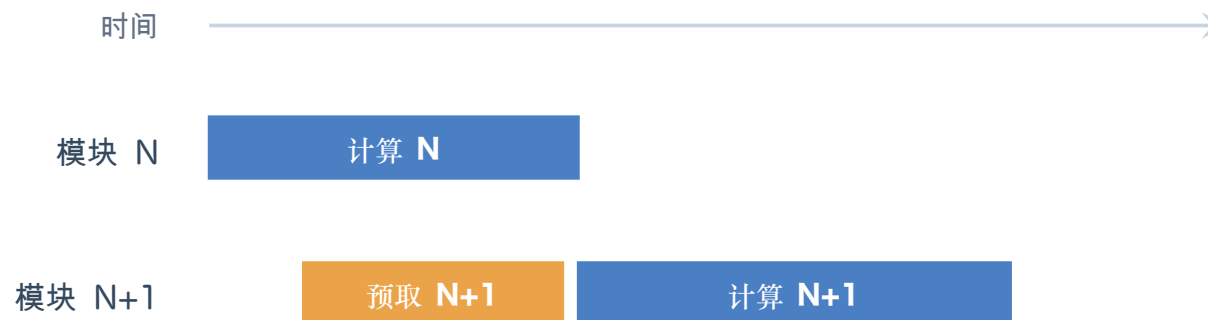
访存消耗内存带宽，计算占用算力资源，
两类资源相互独立，可以并行执行

思路

通过在当前层计算时，预取下一层权重
到 Cache 中，将访存延迟隐藏在计
算时间中

效果

qkv、o、gate_up、down 等模块
权重均可以被预取



执行模块 N 的同时，提前把模块 N+1 的权重读取到 Cache 中，访存与计算并行

Decode 显存墙拆解与优化

问题一：显存容量墙

问题二：访存带宽墙

并发受限

上下文长度受限

模型规模受限

decode 延迟高

局部异构并行

对权重较大的模块进行局部 TP 切分，降低权重占比与访存压力

DCP

按序列维将 KV Cache 分摊到多卡，支持更长上下文

量化

压缩权重与 KV 体积，同时缓解 HBM 容量压力与访存压力

Weight Prefetch

在当前层计算时预取下一层权重到 Cache 中，将访存延迟隐藏在计算时间中



Part 04

展望



vLLM Ascend: 技术持续演进

◆ EEP 容错框架

弹性执行引擎：节点故障自动检测与恢复
检查点机制：毫秒级状态保存与快速恢复
冗余计算：关键任务多副本执行保障
健康监控：实时 NPU 状态监控与预警

◆ 动态 CP 并行

弹性上下文并行：运行时动态调整 CP 规模
负载均衡：根据请求负载自动分配资源
无感扩缩容：服务不中断调整并行度

◆ AFD 完善和优化

单边通信算子支持：优化 Put/Get 原语，降低通信延迟
Proxy 能力完善：增强请求代理与路由调度能力
Connector 优化：改进连接器性能与稳定性
故障检测增强：提升检测精度与响应速度

◆ RL 确定性计算

强化学习支持：为 RLHF 训练提供确定性保障
可复现性：相同输入保证完全一致输出
分布式同步：多节点间计算结果严格一致
精度保障：消除浮点运算非确定性影响

◆ 面向 Agent 的 KV Cache 优化

长对话优化：针对多轮对话场景的 KV 缓存策略
增量更新：仅更新变化的 KV 块节省带宽
智能淘汰：基于注意力机制的缓存淘汰算法
跨请求共享：相同上下文的 KV 缓存复用



vLLM Ascend: 做最好用的昇腾推理引擎



1. 易用性与质量提升

◆ 持续优化易用性

- 官方文档: 及时校准和更新, 保持与代码同步
- 配置简化: 移除冗余环境变量和配置项
- CI 覆盖: 全面覆盖所有功能, 保证版本无 regression

2. 性能与显存优化

◆ 性能进一步释放

- **Hybrid Attention:** 针对新型模型结构的性能优化
- 显存管理: 进一步优化显存分配和回收策略
- 量化优化: 支持更多量化场景下的性能提升

3. 安装简化

◆ 更简单的安装体验

- 二进制包: 发布针对不同硬件的预编译包
- 多版本支持: 适配不同 Python 版本和系统架构
- 源码编译: 优化编译流程, 降低编译失败率

4. 主流模型优化

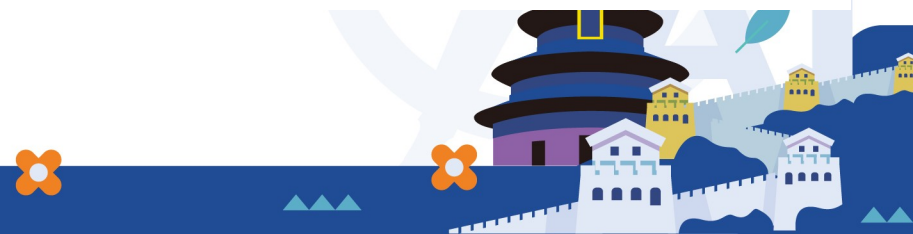
◆ 全面优化支持

- 高质量: 适配最新版本, 确保推理质量
- 高性能: 提升推理效率
- 及时性: 主流模型 Day0 支持

v0.18.0 正式版本即将发布

2026Q1 包括 Qwen3.5 、 GLM5 、 MiniMax2.5 、 Kimi-K2.5 等模型的全面优化

敬请期待



Thanks.

